KC87 Pico Recorder

von Thorsten Kattanek, letzte Änderung 25.10.2026

Inhaltsverzeichnis

Vorwort....2
Was ist der „KC87 Pico Recorder“ ?....2
Technisches „Bla Bla“....3
Wie sieht das Signal aus wenn es per „CSAVE“ aus dem KC87 kommt?....3
Wie können wir das Signal so umwandeln das wir es mit dem Pico empfangen und auswerten können?....4
Wie kommen wir an die Bits und Bytes?....4
Wie kommen die Daten zurück in den KC87?....6
Wie lassen sich WAV-Dateien abspielen?....7

Vorwort

Das Projekt entstand aus meinen Nachforschungen zum KC87. Ich habe mir 2025 einen KC87 zugelegt. Auch recht Preiswert, mit 100€ zu dieser Zeit. So oft findet man diese leider nicht mehr. Die Community ist natürlich deutlich kleiner als die der Commodore Liebhaber. Aber es gibt doch ein paar nette Internetseiten die ich hier mal Aufzähle.

https://hc-ddr.hucki.net/wiki/doku.php
https://www.sax.de/~zander/
https://floooh.github.io/virtualkc/

Das sind schon recht gute Quellen wenn man sich mit alten DDR Computern beschäftigen möchte. In dieser Dokumentation werde ich auch alle Technischen Details was die Elektronik und auch die Programmierung der Firmware und auch der Software für den PC wenn entspr. Tools von mir erstellt werden behandeln. Ich hoffe das am Ende ein nützliches, kleines Stück Hardware raus kommt womit man den KC87 erweitern kann. Am Ende würde ich dieses Projekt dann, mit in das ThePicoRetroTape (https://github.com/ThKattanek/the_pico_retro_tape) Projekt mit einfließen lassen. Aber das erst dann, wenn hier alles so funktioniert wie ich mir das vorstelle.

Was ist der „KC87 Pico Recorder“ ?

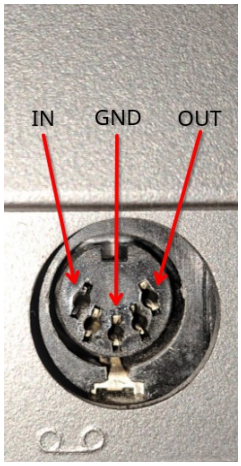
Mit dem Pico Recorder soll am Ende der „alte“ Kassettenrecorder ersetzt werden. Wie der Name schon erahnen lässt, basiert der „neue“ Recorder auf den Raspberry Pi Pico Borad.

Technisches „Bla Bla“

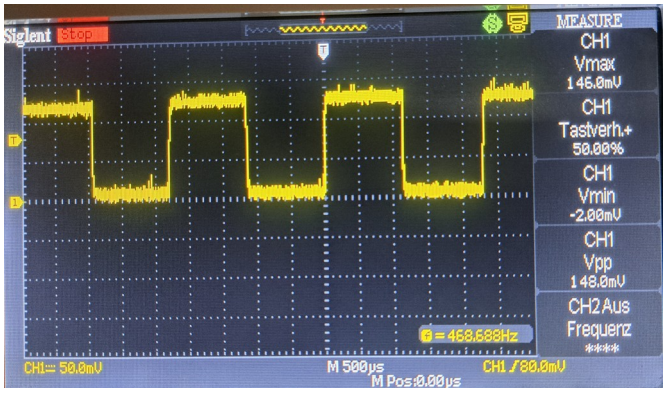
Wie sieht das Signal aus wenn es per „CSAVE“ aus dem KC87 kommt?

Das Kassetten Signal wird aus der rechten Seite des KC87 entnommen. Dort befindet sich eine Buchse wo man früher den Kassettenrecorder angeschlossen hat. Hier ein Bild wo man genau sieht wie diese Buchse standardmäßig belegt ist.



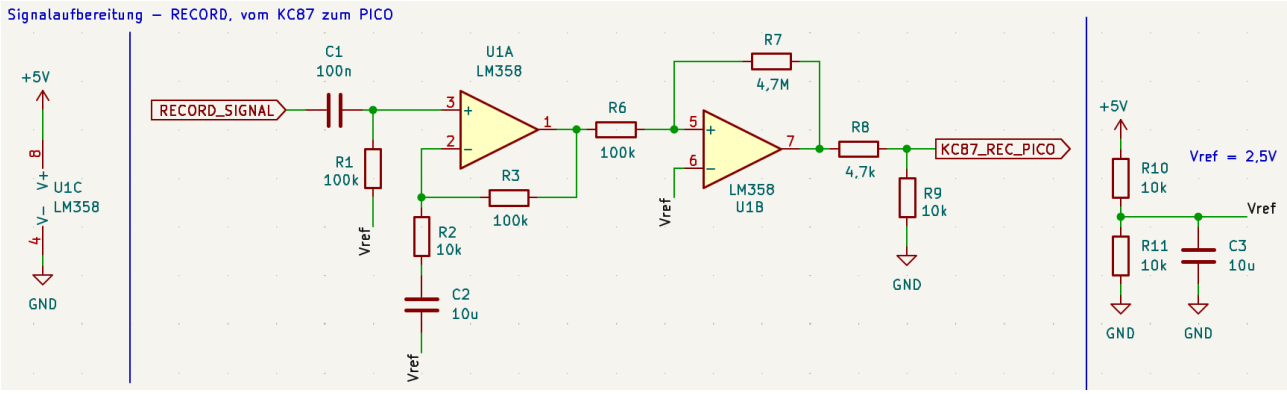


Das Signal was bei OUT raus kommt ist ein Rechtecksignal mit einer Amplitude von ca. 130-150mV. Laut Robotron ist das eine Frequenzmodulation bzw. Frequenzumtastung (FSK). Das ganze sieht mit dem Oszi so aus. Dieses Signal wird so ausgegeben wenn ich „CSAVE“ gleich nach dem Start eingeben und 2x mit Enter bestätige. Das ist nicht im BASIC ! Hier bleibt das Signal dauerhaft an.



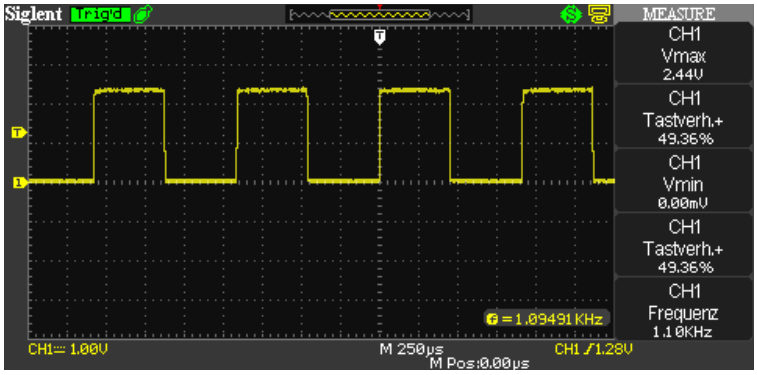
Wie können wir das Signal so umwandeln das wir es mit dem Pico empfangen und auswerten können?

Ich bin da sehr optimistisch und habe mich für folgende Schaltung entschieden. Das Signal vom KC87 ist sehr klein (ca. 130-150mV). Ich verstärke es deshalb zuerst mit einem Operationsverstärker und bringe es danach mit einem zweiten als Komparator auf einen Logik Level von 3,3V, um direkt an den Pico zu gehen. Beide stecken im LM358 (U1) und werden mit 5V VBUS versorgt.



Hier eine kurze Beschreibung der Schaltung:

Das Signal aus dem KC87 kommt über C1 (100n) auf den Eingang von U1A. Mit R1 und der Referenzspannung Vref = 2,5V (Spannungsteiler R10/R11, geglättet durch C3) liegt das Signal in der Mitte der Versorgungsspannung. U1A verstärkt es für das Wechselsignal um den Faktor 11 (1 + R3/R2). U1B arbeitet als Komparator gegen Vref. Die Hysterese über R6/R7 unterdrückt Rauschen. Der Ausgang wird durch den Spannungsteiler R8/R9 auf den Pegel des Pico angepasst. Ein Abgleich ist nicht nötig.
Hier das Ausgangssignal „KC87_REC_PICO“.



Gemessen am Pin „KC87_REC_PICO“ während des Vortons: Vmax = 2,44V, Vmin = 0V, Tastverhältnis ca. 49 %, Frequenz ca. 1,1 kHz.

Wie kommen wir an die Bits und Bytes?

Jetzt wo wir ein vernünftige Digitales Signal haben, können wir uns nun anschauen wie in diesem Datenstrom die Daten codiert sind.

Der Pico misst dazu mit seiner PIO-Einheit die Dauer jeder einzelnen High- und Low-Phase des Signals „KC87_REC_PICO“ am Eingang GPIO 2. Die Messwerte (Auflösung 0,2 µs) werden über USB an den PC geschickt und dort vom Programm „kc87tool“ ausgewertet. Die Aufnahme ist eine Tabelle mit den Pulsdauern, aus der sich Frequenzen und Bits ablesen lassen.

In einer echten „CSAVE“-Aufnahme findet man genau drei verschiedene Pulslängen. Immer zwei gleich lange Pulse (High und Low) bilden eine Periode:

Bit 0: 2 Pulse mit je ca. 250 µs, das sind ca. 2 kHz.

Bit 1: 2 Pulse mit je ca. 455 µs, das sind ca. 1,1 kHz.

Trennzeichen: 2 Pulse mit je ca. 870 µs, das sind ca. 575 Hz.

Die Sollwerte des KC87 liegen etwas höher: Trennzeichen 600 Hz, 1-Bit 1200 Hz und 0-Bit 2400 Hz (laut hc-ddr.hucki.net, Kassetteninterface). Meine gemessenen Werte liegen alle 5 bis 16 % darunter. Die Wiki-Tabelle ordnet die Bits dabei genau umgekehrt zu (0-Bit = 1200 Hz). In meinen Aufnahmen ist die längere Periode das 1-Bit, nur so ergeben sich gültige Prüfsummen und die Kennung D3 D3 D3.

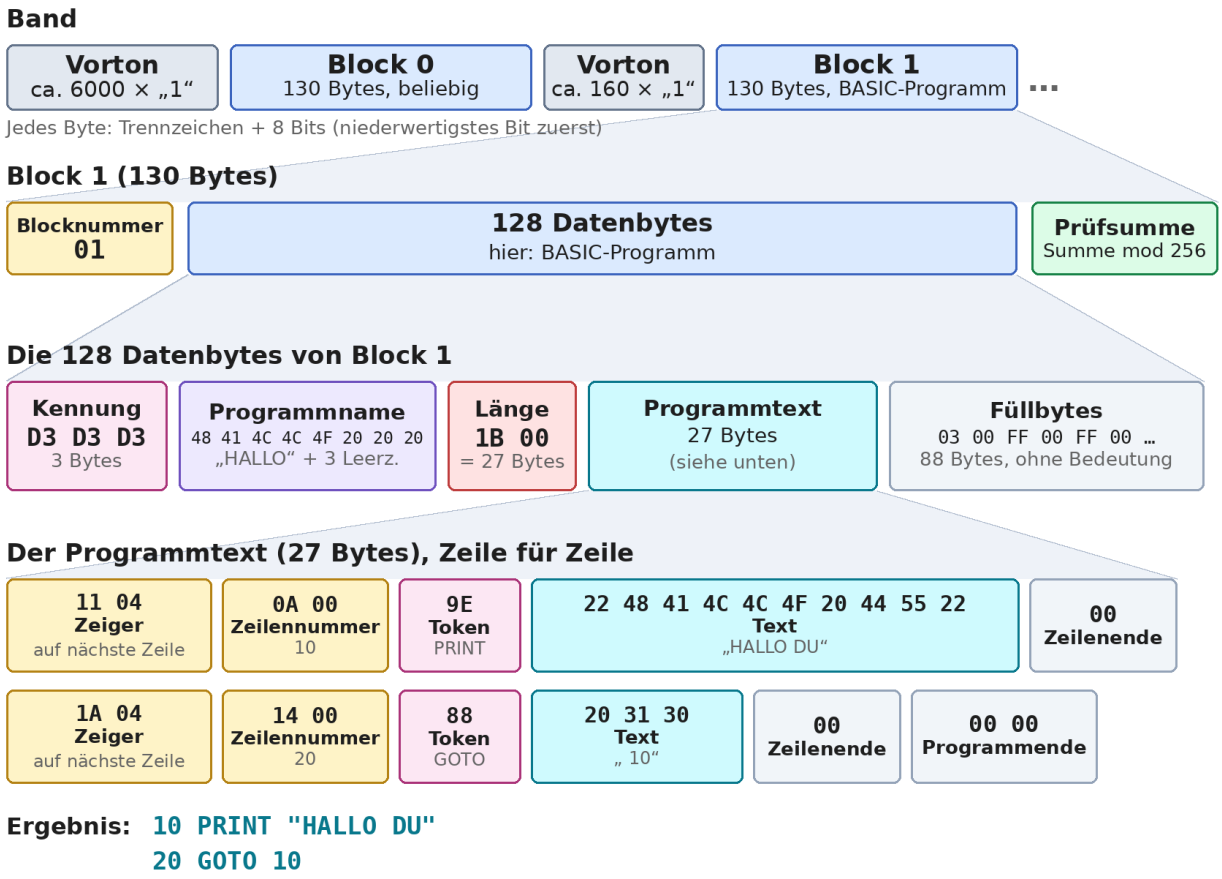
Ein Byte besteht aus 8 Bits, das niederwertigste Bit kommt zuerst. Vor jedem Byte steht ein Trennzeichen. Am Anfang jedes Blocks steht ein Vorton aus lauter 1-Bits, vor dem ersten Block ist er mit ca. 6000 Perioden besonders lang.

Die Daten sind in Blöcken zu 130 Bytes organisiert: Zuerst kommt die Blocknummer, danach 128 Datenbytes und zum Schluss eine Prüfsumme. Die Prüfsumme ist die Summe der 128 Datenbytes modulo 256. Bei meiner Aufnahme stimmten die Prüfsummen beider Blöcke.

Ein mit „CSAVE“ gespeichertes BASIC-Programm beginnt mit den drei Bytes D3 D3 D3 (D3 = BASIC-Programm; D4 steht für eine Datenfeld-Datei und D5 für ein ASCII-Listing). Danach folgt der Programmname mit 8 Zeichen und dann der BASIC-Programmtext. Bei einem Test mit dem Namen „HALLO“ stand dort 48 41 4C 4C 4F 20 20 20, also „HALLO“ mit drei Leerzeichen am Ende.

Das Programm „kc87tool“ macht diese Auswertung mit dem Befehl „decode“: Es zeigt für jeden Block die Nummer und ob die Prüfsumme stimmt, bei BASIC-Programmen auch den Namen. Mit der Option „--tap“ schreibt es die Blöcke mit gültiger Prüfsumme in eine TAP-Datei. Diese TAP-Datei habe ich im Emulator JKCEMU (Version 0.9.9.1) geladen und das Programm lief. Getestet habe ich bisher nur ein BASIC-Programm. Maschinencode-Dateien habe ich noch nicht ausprobiert.

Hier noch einmal der Aufbau der Datei, die ich beim „CSAVE“ empfange. Als Beispiel dient ein Programm mit dem Namen „HALLO“ und den beiden Zeilen 10 PRINT "HALLO DU" und 20 GOTO 10:



Ergebnis:
```
10 PRINT "HALLO DU"
20 GOTO 10
```

Block 0 enthält in meinem Beispiel fast nur Füllbytes und ein paar Reste. Das ist kein Fehler: Ab dem KC 87.2x darf ein BASIC-Programm am Anfang einen beliebigen Block 0 haben, der keine Bedeutung hat. Das Programm steckt in Block 1 (BASIC-Programme beginnen mit Block 1).

Die Länge (1B 00 = 27) zählt den Programmtext mit den beiden Endebytes 00 00. Der Zeiger am Anfang jeder Zeile zeigt auf die nächste Zeile, es ist eine Adresse im Speicher des KC87. Befehle wie PRINT (9E) und GOTO (88) sind als einzelnes Byte, als sogenanntes Token, gespeichert. Alles nach dem Programmende bis zum Blockende sind Füllbytes. Wie ein Programm aussieht, das länger als 128 Bytes ist, habe ich noch nicht getestet.

Wie kommen die Daten zurück in den KC87?

Für das Abspielen gibt der Pico das Signal an GPIO 3 aus („PLAY_SIGNAL_PICO“). Das ist ein Rechtecksignal mit 3,3 V, der KC87 braucht aber deutlich weniger. Laut Bedienungsanleitung hat sein Eingang einen Widerstand von 10 kΩ und braucht mindestens 24 mV Spitze-Spitze. Ein Kassettenrecorder liefert nach der Norm mindestens 200 mV. Eine Obergrenze nennt die Anleitung nicht, deshalb habe ich mich an ca. 200 mV orientiert.

Die Schaltung ist ein einfacher Spannungsteiler: Vom GPIO geht es über R4 (1k8) und den Einstellwiderstand RV2 (1k) auf den Ausgang „PLAY_SIGNAL“, und dort über R5 (150 Ω) nach

Masse. Mit RV2 lässt sich der Pegel am KC87 (10 kΩ Last) rechnerisch von ca. 165 bis 250 mV einstellen, die Quellimpedanz bleibt unter 150 Ω. Ich habe ihn am Oszilloskop auf 220 mV eingestellt. In der Praxis ließ er sich nur bis 204 mV absenken, der Bereich ist also etwas kleiner als berechnet.

Zum Einstellen gibt es die Test-Firmware „kc87_play_test“. Sie gibt an GPIO 3 ein Rechtecksignal mit 1200 Hz, 2400 Hz oder 600 Hz aus (die Sollwerte der Bit-Frequenzen), auf Wunsch im Wechsel.

Für das eigentliche Abspielen schickt der PC die Pulsdauern an den Pico. Ein PIO-Programm gibt sie mit einer Auflösung von 0,2 µs aus. Der Pico sammelt zuerst 2048 Pulse in einem Puffer, bevor die Ausgabe beginnt. Ist der Puffer voll, bremst die USB-Verbindung den PC. Am Ende bleibt der Ausgang auf Low.

Das Programm „kc87tool“ erzeugt die Pulse mit dem Befehl „encode“ aus einer TAP-Datei: Vor jedem Block steht ein Vorton aus 1-Bits (6000 Perioden vor dem ersten, 160 vor jedem weiteren Block). Danach folgt für jedes der 130 Bytes ein Trennzeichen und 8 Bits, das niederwertigste zuerst, und nach dem letzten Byte noch ein Trennzeichen. Die Pulsdauern habe ich aus meiner echten Aufnahme übernommen. Zur Kontrolle habe ich die erzeugten Pulse wieder mit „decode“ gelesen: Das Ergebnis war identisch mit der ursprünglichen TAP-Datei. Der Befehl „play“ schickt die Pulse dann an den Pico.

Eine Stolperfalle: Der KC87 schließt einen Block erst mit der nächsten Flanke ab. Mein Signal hörte beim ersten Versuch nach dem letzten Trennzeichen einfach auf (zwischen zwei Blöcken folgt sonst immer der nächste Vorton). Der KC87 wartete weiter auf Daten. Erst beim zweiten Abspielen meldete er sofort „FILE FOUND“, weil der Vorton am Anfang die fehlende Flanke lieferte. Seit „encode“ nach dem letzten Block einen kurzen Vorton (160 Perioden) anhängt, klappt es gleich beim ersten Mal.

Getestet habe ich es so: Am KC87 „CLOAD"HALLO"“ eingeben und mit Return bestätigen, dann „kc87tool play“ starten. Der KC87 meldet „FILE FOUND“, und das Programm lässt sich mit „RUN“ starten. Damit ist der ganze Weg KC87 → Pico → PC → TAP → Pico → KC87 mit einem kurzen BASIC-Programm geschlossen. Ein längeres Programm mit mehreren Blöcken habe ich im nächsten Abschnitt getestet. Maschinencode habe ich noch nicht ausprobiert.

Wie lassen sich WAV-Dateien abspielen?

Der Pico kann auch Aufnahmen abspielen, die nicht von meinem KC87 stammen, zum Beispiel WAV-Dateien von digitalisierten Kassetten. Dafür liest „kc87tool“ die WAV-Datei ein und macht daraus dieselbe Pulsfolge, die der Pico sonst bei einer Aufnahme misst (Befehl „fromwav“). Der Befehl „play“ akzeptiert WAV-Dateien auch direkt.

Das Programm sucht in der WAV die Nulldurchgänge des Signals. Damit kleine Störungen keine falschen Flanken auslösen, muss das Signal dafür ca. 10 % der Spitze überschreiten (Hysterese). Gleichanteile werden entfernt. Weil die Daten in den Pulsdauern stecken und nicht im Pegel, ist die Lautstärke der Aufnahme egal, und auch invertierte Signale funktionieren. Die Zeitpunkte der Nulldurchgänge werden zwischen zwei Abtastwerten linear berechnet. Dadurch bleiben die Pulsdauern auch bei niedrigen Abtastraten genau genug. Ich habe aus meiner eigenen Pulsfolge

WAV-Dateien mit 44,1, 22,05 und 11,025 kHz erzeugt und wieder eingelesen, die Daten waren jedes Mal identisch. Auch eine künstlich verrauschte, invertierte 16-Bit-Stereo-Datei wurde fehlerfrei gelesen.

Als echten Test habe ich die Datei AIRWOLF.wav benutzt (22,05 kHz, 8 Bit, ca. 80 s). Mit „decode“ fand das Programm 72 Blöcke mit gültigen Prüfsummen: ein BASIC-Programm mit dem Namen „AIRWOLF“ in den Blöcken 1 bis 71 und am Ende einen Block 255, den die Wiki-Seite ebenfalls beschreibt. Am KC87 habe ich „CLOAD"AIRWOLF"“ eingegeben und die WAV-Datei mit „kc87tool play“ über den Pico abgespielt. Das Programm wurde komplett geladen.

Am Ende der Pulsfolge hängt „play“ bei WAV-Dateien einen kurzen Vorton an, aus dem gleichen Grund wie bei TAP-Dateien: Der KC87 schließt den letzten Block erst mit einer weiteren Flanke ab. Mit „fromwav“ und danach „decode --tap“ lässt sich aus der WAV-Datei auch eine TAP-Datei erzeugen, die man im Emulator JKCEMU laden kann.